The Essential of ML: Chapter 2

From Representations to Decisions, cut down to what the professor asks: the Bayes rule, the threshold, the reject band, the loss that picks the summary, cross-entropy as likelihood, and the geometry kNN inherits

Fundamentals of Machine Learning

2026-10-30

How to read this document

This is Lecture 2 with everything removed that the exercise sheet and Tutorial 1 do not test. Six sections in concept order, each ending in a *Tutorial drill* box that holds the professor's own questions (cited as "Tutorial 1, Q2(c)" or "Exercises 2, Q3") and one new question in the same style, with the answer written the way the rubric wants it: claim, mechanism, number or derivation, consequence. Every term is defined before it is used, so read it in order. Left out on purpose: the symbol table, the Lagrange-interpolation proof, the softmax-shift identity, the curse-of-dimensionality radius formula and the full cross-validation protocol; the chapter's notation is kept throughout.

1 Representation and Unavoidable Ambiguity

Object, features, representation, input space

A learner never receives a house or a network flow, only numbers. Two human choices turn an object into data: **describe** it (select properties: area, age, duration, port) and **encode** the description as a vector $x = [x_1, \ldots, x_d]^\top \in \mathcal{X} \subseteq \mathbb{R}^d$, the **representation**; $\mathcal{X}$ is the **input space**, d the number of **features**. A numerical property is one component; a categorical property with K values becomes K binary components with a single 1 (**one-hot encoding**). The map from object to vector is written $x = g(z)$.

A house of 145 m^2, 4 rooms, 12 years, neighbourhood A among $\{A, B, C\}$ is $x = [145, 4, 12, 1, 0, 0]^\top \in \mathbb{R}^6$. Coding the neighbourhoods as $1, 2, 3$ instead would invent an order ("B is halfway between A and C") that every distance-based method would then believe; one-hot puts all three at the same mutual distance $\sqrt{2}$. The encoding step decides what "similar" means.

Why the representation puts a floor under every classifier

Write X and Y for the input and output regarded as random, $\mathbb{P}(X, Y)$ for their **joint distribution** (how likely each pair is in the world) and $\mathbb{P}(Y \mid X = x)$ for the **conditional distribution**: what is still uncertain about Y once the input is known to be x. If two physically different objects z_A (label 0) and z_B (label 1) satisfy $g(z_A) = g(z_B) = x$, then $\mathbb{P}(Y \mid X = x)$ is genuinely spread over both labels. Any deterministic classifier h outputs one label $h(x)$ for both objects and is wrong on one of them. The information that separated them was destroyed by g; no learner downstream can recover it. The smallest achievable error at x, $1 - \max_c \mathbb{P}(Y = c \mid X = x)$, is the **Bayes error** at x: a property of the problem and its representation, not of any model.

Key insight. More features are not automatically better. A feature U with $\mathbb{P}(Y \mid X, U) = \mathbb{P}(Y \mid X)$ (irrelevant given X) leaves the Bayes error unchanged and still makes learning from finite data harder: it adds a coordinate in which neighbours differ at random and parameters that must be estimated from the same N examples.

Tutorial drill

Exercises 2, Q1(a)–(b). *Two events z_A (benign) and z_B (malicious) have the same recorded vector x. Why can no deterministic classifier using only x classify both correctly? If 30% of events represented by x are malicious, give the Bayes decision and its conditional error under equal costs.*
Claim: it is impossible, and the best error at x is 0.3. Mechanism: a deterministic classifier is a function of its input; both events present the same input, so both receive $h(x)$, and the labels differ. Number: $\mathbb{P}(Y = 1 \mid x) = 0.3$, $\mathbb{P}(Y = 0 \mid x) = 0.7$; with equal costs the Bayes rule declares the more probable class, **benign**, and is wrong with probability 0.3; declaring malicious would be wrong with probability 0.7. Consequence: the 0.3 belongs to the representation g, not to the model; only a new feature that splits the two events can lower it.
Exercises 2, Q1(c). *Propose a feature that might separate them; when is it legitimate and when is it leakage?* The destination port, the payload-size distribution or the process that opened the connection. Legitimate when it is available in the same form at decision time for new events; leakage when it is produced by the labelling itself or known only afterwards ("matched a signature written after the incident", "host appears in the incident report"): such a feature gives a perfect training score and nothing at deployment.
New. *The Bayes error is 0 at every x. Does that mean a classifier trained on 1000 examples will have error 0?* No. Bayes error 0 says the representation carries enough information; it says nothing about whether the learner can find the rule from 1000 examples. A 1-NN classifier in $d = 50$ dimensions on 1000 points, or a linear model when the true boundary is a circle, has error far above 0. The Bayes error is the floor; the gap above the floor is the subject of Sections 4–6.

2 Decisions When the Distribution Is Known: the Bayes Rule

Action, decision loss, conditional risk, Bayes decision rule

Let $\mathcal{A}$ be the **action space** (numbers to report, classes to declare, "send to a human"). The **decision loss** $\ell_{\text{dec}}(y, a)$ is the cost of taking action a when the truth is y: squared error $(y-a)^2$, absolute error $|y-a|$, zero–one loss $\mathbf{1}(y \neq a)$, or a cost table. For a fixed input x the **conditional risk** of action a is its average damage under what we know,

$$R(a \mid x) = \mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x] = \sum_{y} \ell_{\text{dec}}(y, a)\, \mathbb{P}(Y = y \mid X = x),$$

and the **Bayes decision rule** picks, at every x separately, the action with the smallest conditional risk: $f^*(x) \in \arg\min_{a \in \mathcal{A}} R(a \mid x)$. "Bayes" means optimal under a probability distribution; no prior on parameters is involved.

Pointwise is enough: by the tower property $\mathbb{E}[\ell_{\text{dec}}(Y, f(X))] = \mathbb{E}[\mathbb{E}[\ell_{\text{dec}}(Y, f(X)) \mid X]]$, and the inner average is minimised at every x by $f^*(x)$, so the outer average is minimised too. The value $\mathbb{E}[\ell_{\text{dec}}(Y, f^*(X))]$ is the **Bayes risk**, the floor of Section 1 averaged over inputs.

Zero–one loss, then unequal costs

Zero–one. $R(a \mid x) = \mathbb{E}[\mathbf{1}(Y \neq a) \mid x] = 1 - \mathbb{P}(Y = a \mid x)$, so $f^*(x) = \arg\max_c \mathbb{P}(Y = c \mid X = x)$, the most probable class, with conditional risk $1 - \max_c \mathbb{P}(Y = c \mid x)$. This is the only loss for

which "predict the most likely class" is right.

Binary detection with costs. Class 1 is the event to detect, $p(x) = \mathbb{P}(Y = 1 \mid X = x)$, correct decisions cost 0, a false positive costs C_{FP}, a false negative C_{FN}. Declaring positive is wrong exactly when $Y = 0$; declaring negative exactly when $Y = 1$:

$$R(\text{positive} \mid x) = C_{\mathrm{FP}}\,[1 - p(x)], \qquad R(\text{negative} \mid x) = C_{\mathrm{FN}}\,p(x).$$

Positive is cheaper iff $C_{\mathrm{FP}}(1-p) < C_{\mathrm{FN}}\,p \iff C_{\mathrm{FP}} < (C_{\mathrm{FP}} + C_{\mathrm{FN}})\,p$, that is

$$\boxed{\text{declare positive iff } p(x) > \frac{C_{\mathrm{FP}}}{C_{\mathrm{FP}} + C_{\mathrm{FN}}}}$$

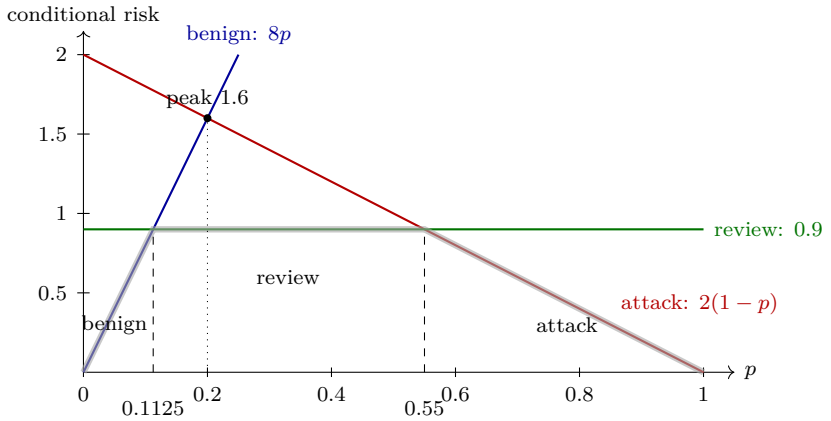
Equal costs give $\frac{1}{2}$. C_{FP} sits on top: expensive misses (C_{FN} large) give a *small* threshold and more alarms; expensive false alarms push it up ("beyond reasonable doubt").

A third action: review, and why one threshold stops being enough

Add "send to a human", which resolves the case correctly at a fixed cost C_{R} whatever the truth: $R(\text{review} \mid x) = C_{\mathrm{R}}$, a horizontal line. Each action is optimal where its line is lowest:

$$\begin{aligned} \text{negative iff } C_{\mathrm{FN}}\,p \le C_{\mathrm{R}} \text{ and } C_{\mathrm{FN}}\,p \le C_{\mathrm{FP}}(1-p) &\iff p \le C_{\mathrm{R}}/C_{\mathrm{FN}}, \\ \text{positive iff } C_{\mathrm{FP}}(1-p) \le C_{\mathrm{R}} \text{ and } C_{\mathrm{FP}}(1-p) \le C_{\mathrm{FN}}\,p &\iff p \ge 1 - C_{\mathrm{R}}/C_{\mathrm{FP}}, \\ \text{review iff } C_{\mathrm{R}}/C_{\mathrm{FN}} < p < 1 - C_{\mathrm{R}}/C_{\mathrm{FP}} & \end{aligned}$$

(the simplifications hold whenever the band exists; the second condition in each line is then implied). Two lines cross once, so two actions need one threshold; a third line of constant height cuts the lower envelope $\min\{C_{\mathrm{FN}}p, C_{\mathrm{FP}}(1-p)\}$ twice, so three actions need two. The envelope peaks at $p^* = C_{\mathrm{FP}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$ with height $C_{\mathrm{FP}}C_{\mathrm{FN}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$: review is ever used iff C_{R} is below that peak. Raising C_{R} moves both boundaries inward and the band shrinks to nothing at the peak; cheaper review widens it.



Tutorial 1, Q2: $C_{\mathrm{FP}} = 2$, $C_{\mathrm{FN}} = 8$, $C_{\mathrm{R}} = 0.9$. The grey path is the lower envelope, the risk of the best action at each p.

Three classes and a cost table: the most probable class loses

Triage: healthy, mild, severe with probabilities $(0.5, 0.3, 0.2)$; actions discharge, monitor, treat; costs (rows truth, columns action): healthy $(0, 1, 3)$, mild $(4, 0, 2)$, severe $(20, 6, 0)$. Conditional risks are dot products of the probability row with each column: discharge $0.5(0) + 0.3(4) + 0.2(20) = 5.2$; monitor $0.5(1) + 0.3(0) + 0.2(6) = 1.7$; treat $0.5(3) + 0.3(2) + 0 = 2.1$. Bayes decision: **monitor**, although "healthy" is the most probable state. The 20 in the corner rules

the calculation.

Tutorial drill

Tutorial 1, Q2(a)–(b). $C_{\text{FP}} = 2$, $C_{\text{FN}} = 8$. *Derive the conditional risks and the threshold; apply to* $p = 0.08, 0.35, 0.85$.
$R(\text{attack} \mid x) = 2(1-p)$, $R(\text{benign} \mid x) = 8p$; attack iff $2(1-p) < 8p \iff p > 0.2 = 2/10$. At 0.08: 1.84 vs 0.64, benign. At 0.35: 1.30 vs 2.80, **attack**, although the flow is more likely benign, because missing it costs four times more. At 0.85: 0.30 vs 6.80, attack.
Tutorial 1, Q2(c); Exercises 2, Q3. *Review costs* 0.9 *and resolves the case. Derive the three regions.* Benign iff $8p \le 0.9 \iff p \le 0.1125$; attack iff $2(1-p) \le 0.9 \iff p \ge 0.55$; review on $(0.1125, 0.55)$. Check at $p = 0.3$: benign 2.4, attack 1.4, review 0.9. The band exists because the envelope peaks at $p = 0.2$ with value $1.6 > 0.9$. The 0.35 flow now goes to review. With the exercise sheet's numbers ($C_{\text{FP}} = 1$, $C_{\text{FN}} = 9$, $C_{\text{R}} = 0.6$): benign iff $p \le 0.6/9 = 0.067$, attack iff $p \ge 0.4$, review between; the peak is 0.9 at $p = 0.1$, so review disappears once $C_{\text{R}} \ge 0.9$.
Tutorial 1, Q2(d). *Why is moving the threshold for costs different from correcting for class imbalance?* The threshold acts at the *decision* stage: it trusts $p(x)$ and asks what to do with it. Imbalance corrections act at the *estimation* stage: reweighting or resampling helps fit a posterior from few minority examples, and a prior-shift correction changes the posterior itself. A threshold cannot repair a badly estimated p; reweighting does not encode what a missed attack costs.
New. *Give a cost ratio for which the zero–one classifier and the cost-sensitive one agree at* $p(x) = 0.999$*, and one for which they disagree at* $p(x) = 0.3$. They agree whenever the threshold stays below p: any $C_{\text{FP}}/(C_{\text{FP}} + C_{\text{FN}}) < 0.999$, e.g. $C_{\text{FP}} = C_{\text{FN}}$. They disagree at 0.3 whenever the threshold drops below 0.3: $C_{\text{FN}} > \frac{7}{3} C_{\text{FP}}$, e.g. $C_{\text{FP}} = 1$, $C_{\text{FN}} = 3$ gives threshold $0.25 < 0.3$ and the cost-sensitive rule alarms while the zero–one rule stays quiet.

3 The Loss Decides the Best Point Prediction

Mean, median, quantile

For a numerical output and a fixed x, write $F(y \mid x) = \mathbb{P}(Y \le y \mid X = x)$ for the conditional distribution function and $p(y \mid x)$ for its density. The **conditional mean** is $\mathbb{E}[Y \mid X = x]$; a **conditional median** is any m with $F(m \mid x) = \frac{1}{2}$ (more generally $\mathbb{P}(Y \le m) \ge \frac{1}{2}$ and $\mathbb{P}(Y \ge m) \ge \frac{1}{2}$); a **conditional τ-quantile** is any m with $F(m \mid x) = \tau$. The question "what is the best prediction?" has no answer until ℓ_{dec} is specified, because each loss selects a different one of these summaries.

Three derivations, one method: differentiate the conditional risk in a

Squared loss → mean. Write $m = \mathbb{E}[Y \mid x]$ and $Y - a = (Y - m) + (m - a)$. Squaring and taking the conditional expectation, the cross term $2(m-a)\mathbb{E}[Y - m \mid x]$ vanishes because $\mathbb{E}[Y - m \mid x] = 0$, so

$$\mathbb{E}[(Y-a)^2 \mid x] = \text{Var}(Y \mid x) + (m-a)^2,$$

unavoidable spread plus your own offset squared; minimised uniquely at $a = m$.
Absolute loss → median. $\mathbb{E}|Y - a| = \int_{-\infty}^{a} (a-y)\,p(y)\,dy + \int_{a}^{\infty} (y-a)\,p(y)\,dy$. Differentiating in a (the boundary terms vanish because both integrands are 0 at $y = a$): $\frac{d}{da}\mathbb{E}|Y-a| = F(a) - [1 - F(a)] = 2F(a) - 1$, zero iff $F(a) = \frac{1}{2}$; the second derivative $2p(a) \ge 0$ confirms a minimum.
Asymmetric loss → quantile. Let under-prediction ($a < y$) cost c_u per unit and over-prediction cost c_o per unit:

$$\mathbb{E}[\ell(Y,a) \mid x] = c_u \int_a^{\infty} (y-a)\,p(y)\,dy + c_o \int_{-\infty}^{a} (a-y)\,p(y)\,dy, \qquad \frac{d}{da} = -c_u[1 - F(a)] + c_o F(a).$$

Setting this to zero gives $F(a) = c_u/(c_u + c_o)$: the optimal report is the quantile at level $\tau = c_u/(c_u + c_o)$. Expensive under-prediction pushes the report up. With $c_u = c_o$ the median returns; the chapter's ℓ_τ with weights τ and $1 - \tau$ is the case $c_u = \tau$, $c_o = 1 - \tau$.

Exercises 2, Q2: the exponential response time

$p(y \mid x) = \frac{1}{4}e^{-y/4}$ on $y \geq 0$, $F(y \mid x) = 1 - e^{-y/4}$.

- *Squared loss:* $\mathbb{E}[Y \mid x] = \int_0^\infty y\, \frac{1}{4}e^{-y/4}\, dy = 4$ (an exponential with scale β has mean β).
- *Absolute loss:* $F(a) = \frac{1}{2} \iff e^{-a/4} = \frac{1}{2} \iff a = 4 \ln 2 \approx 2.77$.
- *Asymmetric loss* $\ell(y, a) = 4(y - a)$ if $a < y$, $a - y$ if $a \geq y$: $c_u = 4$, $c_o = 1$, $\tau = 4/5$; $1 - e^{-a/4} = 0.8 \iff a = 4 \ln 5 \approx 6.44$. The report is set so the response exceeds it only 20% of the time.

Three losses, three numbers (2.77, 4, 6.44) from one unchanged distribution. The mean exceeds the median because the distribution is right-skewed: squared loss punishes large errors heavily and chases the long tail; absolute loss counts errors linearly and sits at the middle.

Key insight. A model that outputs only $\mathbb{E}[Y \mid x]$ cannot serve a later asymmetric loss: distributions with the same mean 4 can have very different 80th percentiles. Estimate the conditional distribution, or the needed quantile directly (quantile regression with the matching asymmetric loss). Only an extra distributional assumption ("it is exponential") lets the mean determine the quantile.

Tutorial drill

Exercises 2, Q2(a)–(b) is the example above; the rubric wants the derivative $-c_u[1 - F(a)] + c_o F(a) = 0$ written out, not just "the 80th percentile".
Exercises 2, Q2(c). *Why may a conditional-mean model be insufficient when the loss becomes asymmetric? What richer quantity should it estimate?* Claim: the mean does not determine the quantile. Mechanism: the quantile depends on the shape of $p(y \mid x)$, not on its first moment; a narrow distribution around 4 has its 80th percentile near 4, this one at 6.44. Consequence: estimate $p(y \mid x)$ itself, or the conditional quantiles the application needs, each trained with its own asymmetric loss.
New (counterexample). *Give a conditional distribution for which the squared-loss optimum is a value that never occurs, and compute the absolute-loss optimum.* Delivery time Y is 100 w.p. 0.5, 200 w.p. 0.3, 600 w.p. 0.2. Mean $= 50 + 60 + 120 = 230$, a time that never happens; check: risk at 230 is $0.5(130)^2 + 0.3(30)^2 + 0.2(370)^2 = 36{,}100 < 37{,}000$ at $a = 200$. Median: $F(100) = 0.5$ and $F(200) = 0.8$, so every $a \in [100, 200]$ is a median, each with expected absolute loss 130 (at 100: $0.3(100) + 0.2(500)$; at 200: $0.5(100) + 0.2(400)$); at 230 it is 148. The rare 600 drags the mean and leaves the median untouched; the 0.9-quantile is 600.

4 From the Known Distribution to Finite Data

Training data, i.i.d., population risk, empirical risk, plug-in

We never know $\mathbb{P}(X, Y)$. What we observe is a **training dataset** $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$ (superscript (i) indexes examples, subscript j indexes features: $x_j^{(i)}$), modelled as **i.i.d.** draws from $\mathbb{P}(X, Y)$: independent, same distribution. For a rule f the **population risk** $R(f) = \mathbb{E}[\ell(Y, f(X))]$ is the average loss over the whole world (what we want, unknowable); the **empirical risk** $\widehat{R}(f) = \frac{1}{N}\sum_{i=1}^N \ell(y^{(i)}, f(x^{(i)}))$ is the same average over the sample (computable). The **plug-in principle** replaces the unknown $\mathbb{P}(Y \mid X = x)$ in the Bayes rule by an estimate built from $\mathcal{D}$, then applies the decision rule of Section 2 unchanged.

For a fixed f chosen without looking at the data, $\mathbb{E}[\widehat{R}(f)] = R(f)$: each term is an independent copy of $\ell(Y, f(X))$ with mean $R(f)$, so the empirical risk is an honest estimate. The honesty is lost the moment f is *selected* to make $\widehat{R}$ small (Section 5).

k-nearest neighbours: the Bayes rule estimated locally

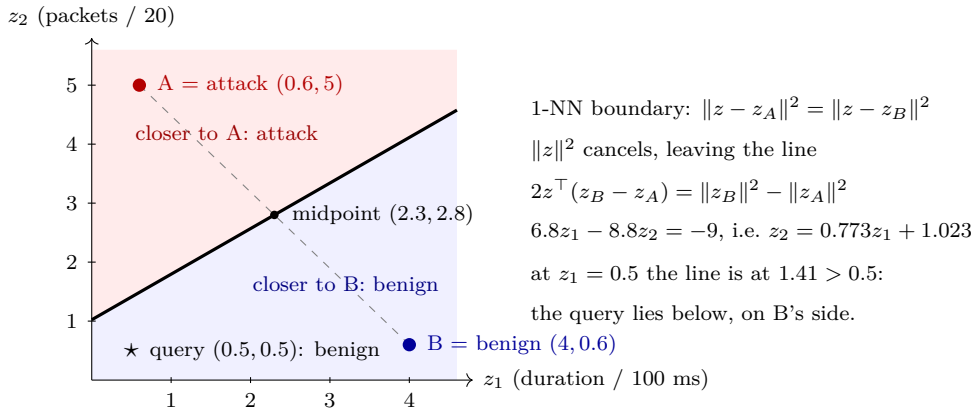
The Bayes rules need $\mathbb{E}[Y \mid X = x]$ or $\mathbb{P}(Y = c \mid X = x)$. If many examples had exactly input x we would average their outputs; exact repeats are rare, so use the k examples whose representations are *closest*. With $N_k(x)$ the indices of the k nearest training inputs under a chosen distance,

$$\hat{f}_k(x) = \frac{1}{k} \sum_{i \in N_k(x)} y^{(i)}, \qquad \hat{p}_k(Y = c \mid x) = \frac{1}{k} \sum_{i \in N_k(x)} \mathbf{1}(y^{(i)} = c),$$

and the kNN classifier under zero–one loss is the **majority vote** $\arg\max_c \hat{p}_k$; under costs, feed $\hat{p}_k$ to the threshold of Section 2. For $k = 1$ the plane is cut into **Voronoi cells**, and the boundary between two training points is their perpendicular bisector. kNN has no parameters; k, the distance and the feature scales are **hyperparameters**, chosen outside the rule (Section 6), because the rule itself would always pick $k = 1$ (training error 0).

Tutorial 1, Q3(a)–(b): the units choose the neighbour

Query flow $x = (50 \text{ ms}, 10 \text{ packets})$; $x_A = (60, 100)$, $x_B = (400, 12)$. Raw Euclidean: $d(x, x_A) = \sqrt{10^2 + 90^2} = \sqrt{8200} \approx 90.6$, $d(x, x_B) = \sqrt{350^2 + 2^2} \approx 350.0$: **A** is nearest, because milliseconds produce the big numbers and 350 ms dwarfs a packet count that is almost identical. Scale duration by 100 and packets by 20: $z = (0.5, 0.5)$, $z_A = (0.6, 5.0)$, $z_B = (4.0, 0.6)$; $d = \sqrt{0.01 + 20.25} \approx 4.50$ to A and $\sqrt{12.25 + 0.01} \approx 3.50$ to B: **B** is nearest. The flows did not change; the metric did. Dividing coordinate j by s_j is the weighted distance $\sqrt{\sum_j (x_j - z_j)^2 / s_j^2}$ on the raw data. Standardise with means and standard deviations fitted on the *training* set and reused for every new input.



Tutorial 1, Q3(c): the 1-NN boundary is the perpendicular bisector through the midpoint $(2.3, 2.8)$.

The curse of dimensionality

kNN needs training points genuinely near the query. Put n uniform points in the unit ball of $\mathbb{R}^d$ and the query at its centre; call a point a true neighbour if it lies within $r = 0.1$. A d-ball of radius r has volume r^d times the unit ball's, so one point is a neighbour with probability 0.1^d, the count $M_d \sim \text{Binomial}(n, 0.1^d)$, $\mathbb{E}M_d = n\,0.1^d$, and

$$\mathbb{P}(M_d \geq 1) = 1 - (1 - 10^{-d})^n \approx 1 - e^{-n10^{-d}}.$$

With $n = 1000$: $d = 2$: ≈ 1; $d = 3$: $1 - e^{-1} = 0.632$; $d = 4$: $1 - e^{-0.1} = 0.095$; $d = 10$: 10^{-7}.

The first dimension where the chance drops below one half is $d = 4$. The "neighbours" kNN then averages are far away, and local estimation loses its justification: this is what motivates a *shared* model whose parameters are learned from all examples at once (Section 5).

Tutorial drill

Tutorial 1, Q3(d). *With $d_w^2(z, z') = w(z_1 - z_1')^2 + (z_2 - z_2')^2$, find the critical w where the nearest flow changes; why choose w by validation?*
$d_w^2(z, z_A) = 0.01w + 20.25$, $d_w^2(z, z_B) = 12.25w + 0.01$; equal when $20.24 = 12.24w$, so $w^* = 1.654$. For $w < 1.654$ B (benign) is nearest, for $w > 1.654$ A (attack). The label depends on how much a unit of duration counts against a unit of packets. w is a hyperparameter the 1-NN rule cannot estimate; tuning it on this query or on test answers fits the metric to the evaluation. Select it on held-out validation flows, assess once on the test set.
Exercises 2, Q5(a)–(b). *Query* $(50{,}000, 30)$*;* $x_A = (51{,}000, 60)$, $x_B = (60{,}000, 31)$. *Nearest raw, then with scales* 10,000 *and* 10. Raw: $d_A = \sqrt{1000^2 + 30^2} \approx 1000.4$, $d_B \approx 10{,}000$: A, a person 30 years older. Scaled: $(5, 3)$, $(5.1, 6)$, $(6, 3.1)$; $d_A = \sqrt{0.01 + 9} = 3.00$, $d_B = \sqrt{1 + 0.01} = 1.00$: B. Same people, different metric. **Q5(d)**: one-hot locations put every pair at distance $\sqrt{2}$, so Euclidean distance assumes all locations are equally different; use coordinates, a hierarchy (neighbourhood < city < region) with a distance table, or a learned embedding, weighted against the other features by validation.
Exercises 2, Q6(b). $n = 1000$*: first d with* $\mathbb{P}(M_d \geq 1) < 0.5$*?* $d = 4$: $\mathbb{E}M_4 = 0.1$, $\mathbb{P}(M_4 \geq 1) = 1 - 0.9999^{1000} \approx 0.095$, against 0.632 at $d = 3$. **Q6(d)**: the calculation flatters kNN: the query sits at the centre (most volume in high d is near the surface, where half the neighbourhood is empty), one neighbour is counted as enough (averaging label noise needs k), and every dimension is assumed relevant (irrelevant ones spend the radius on noise).
New. *What is the training error of* 1*-NN, and why does it not say* 1*-NN is good?* Zero: each training point is its own nearest neighbour. It measures memorisation, not the population risk; with k chosen by training error the rule would always pick $k = 1$, which is why k must be chosen on data the rule did not see.

5 Parameterized Models, ERM and Cross-Entropy

Parameterized model, hypothesis space, parameters versus hyperparameters

A **parameterized model** $f_\theta(x)$ is a function whose behaviour is set by a vector of adjustable numbers $\theta \in \Theta$, the **parameters**; the set of all functions it can produce, $\mathcal{H} = \{f_\theta : \theta \in \Theta\}$, is the **hypothesis space**. Linear regression: $f_\theta(x) = w^\top x + b$, $\theta = (w, b)$; a polynomial $\theta_0 + \theta_1 x + \cdots + \theta_p x^p$: the coefficients are learned, the degree p is chosen before learning. **Hyperparameters** are those outside choices (p, k, the number of layers, the step size); some define $\mathcal{H}$, others the procedure. Choosing $\mathcal{H}$ before seeing data is an **inductive bias**. A probabilistic classifier outputs a vector $\hat{p}_\theta(x) \in \Delta^{C-1}$, the **probability simplex** (nonnegative, sums to one), usually by passing C scores $s_{\theta,c}(x)$ through softmax $p_c = e^{s_c} / \sum_{c'} e^{s_{c'}}$; for two classes this is the sigmoid of the score difference, $\sigma(s) = 1/(1 + e^{-s})$.

Two losses, empirical risk minimisation, and why training risk flatters

The **training loss** ℓ_{train} guides estimation from data; the **decision loss** ℓ_{dec} turns the inferred quantity into an action. They may coincide (squared loss for a regression) or differ (train with cross-entropy, decide with a cost threshold). **ERM** chooses

$$\hat{\theta} \in \arg\min_{\theta \in \Theta} \widehat{R}_{\text{train}}(\theta), \qquad \widehat{R}_{\text{train}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell_{\text{train}}(y^{(i)}, f_\theta(x^{(i)})).$$

Two facts follow from the definition alone. (i) If $\mathcal{H}_1 \subseteq \mathcal{H}_2$ then $\min_{\mathcal{H}_2} \widehat{R} \le \min_{\mathcal{H}_1} \widehat{R}$: a bigger family never fits the training data worse, down to 0 for a degree-$(N-1)$ polynomial through N distinct points. (ii) With θ^* the population minimiser, $\widehat{R}(\hat{\theta}) \le \widehat{R}(\theta^*)$ on every sample, so $\mathbb{E}[\widehat{R}_{\text{train}}(\hat{\theta})] \le R(\theta^*) \le \mathbb{E}[R(\hat{\theta})]$: after minimisation the training risk is *optimistic*. Training data estimate θ inside a fixed configuration; they cannot choose the configuration.

Cross-entropy = maximum likelihood; the logistic-regression gradient

Zero–one training risk $\frac{1}{N}\sum \mathbf{1}(y^{(i)} \neq \arg\max_c s_{\theta,c}(x^{(i)}))$ is a staircase in θ: constant between decision flips (gradient 0), undefined at them, blind to confidence. The **cross-entropy loss** replaces it by the negative log of the probability given to the class that occurred,

$$\ell_{\text{CE}}(y^{(i)}, \hat{p}_\theta(x^{(i)})) = -\log \hat{p}_{\theta,y^{(i)}}(x^{(i)}), \qquad \widehat{R}_{\text{CE}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log \hat{p}_{\theta,y^{(i)}}(x^{(i)}),$$

binary form $\ell_{\text{BCE}}(y,\hat{p}) = -y\log\hat{p} - (1-y)\log(1-\hat{p})$. **Likelihood.** Under i.i.d. sampling the conditional likelihood of the observed labels is $L(\theta) = \prod_i \hat{p}_{\theta,y^{(i)}}(x^{(i)})$ and $\log L(\theta) = -N\,\widehat{R}_{\text{CE}}(\theta)$; log is increasing and $-N < 0$, so minimising empirical cross-entropy *is* maximising the conditional likelihood. **Calibration.** At fixed x with true $p_c(x)$, $\mathbb{E}[-\log \hat{p}_{\theta,Y}(x) \mid x] = -\sum_c p_c \log \hat{p}_{\theta,c} = H(p) + \text{KL}(p \,\|\, \hat{p}_\theta) \ge H(p)$, with equality iff $\hat{p}_\theta(x) = p(x)$ (Gibbs). The floor $H(Y \mid X)$ is the problem's own noise; the excess is the model's divergence from the truth, so cross-entropy pushes the outputs toward the true posterior, which any decision rule of Section 2 can then use.
Logistic regression. $\hat{p}_\theta(x) = \sigma(s)$, $s = w^\top x + b$, $y \in \{0,1\}$. Using $\sigma'(s) = \sigma(s)(1-\sigma(s))$:

$$\frac{\partial \ell_{\text{BCE}}}{\partial s} = -y\,\frac{\sigma(1-\sigma)}{\sigma} + (1-y)\,\frac{\sigma(1-\sigma)}{1-\sigma} = -y(1-\sigma) + (1-y)\sigma = \sigma(s) - y.$$

By the chain rule $\partial s/\partial w = x$ and $\partial s/\partial b = 1$, so

$$\nabla_w \widehat{R}_{\text{CE}} = \frac{1}{N}\sum_{i=1}^{N} (\hat{p}_\theta(x^{(i)}) - y^{(i)})\, x^{(i)}, \qquad \frac{\partial \widehat{R}_{\text{CE}}}{\partial b} = \frac{1}{N}\sum_{i=1}^{N} (\hat{p}_\theta(x^{(i)}) - y^{(i)}).$$

The gradient is the residual "predicted probability minus label" times the input: a confident correct example contributes almost nothing, a confident mistake pushes with weight near ± 1. For C classes the same computation gives $\partial \ell_{\text{CE}}/\partial s_c = \hat{p}_c - \mathbf{1}(c = y)$.

Numbers: Tutorial 1, Q6(a) and Exercises 2, Q4

One example. Score $s = 1.2$: $\hat{p} = 1/(1+e^{-1.2}) = 0.769$. True label 1: loss $-\ln 0.769 = 0.263$; $\partial\ell/\partial s = 0.769 - 1 = -0.231$; with $x = (1,2)$ the gradient in w is $(-0.231, -0.462)$, so a gradient step raises s. With label 0 the loss would be $-\ln 0.231 = 1.466$ and the derivative $+0.769$.
Zero–one and cross-entropy disagree. Probability given to the correct class on four examples: A $= (0.60, 0.60, 0.60, 0.60)$, B $= (0.99, 0.99, 0.99, 0.20)$. Zero–one at threshold 0.5: A 0/4, B 1/4: **A wins**. Cross-entropy: A $= -\ln 0.6 = 0.511$; B $= [3(0.0101) + 1.609]/4 = 0.410$: **B wins**. Zero–one sees only which side of 0.5 each number falls; cross-entropy sees how much probability reached the truth, on a log scale.

Key insight. The disagreement does not make cross-entropy a bad training loss. The training loss *estimates* probabilities, the decision loss is applied *afterwards*; cross-entropy is the loss whose expectation is minimised by the true posterior, zero–one cannot estimate anything (flat, confidence-blind); and probabilities close to the truth give a near-optimal Bayes decision for *every* downstream loss (zero–one, costs, reject band). A's flat 0.6 would be useless the moment costs or review enter.

Tutorial drill

Tutorial 1, Q6(b). *Why is minimising empirical cross-entropy equivalent to maximising the conditional likelihood under i.i.d. sampling?* Independence turns the joint probability of the N observed labels into the product $\prod_i \hat{p}_{\theta,y^{(i)}}(x^{(i)})$; taking logs turns it into $\sum_i \log \hat{p}_{\theta,y^{(i)}}(x^{(i)}) = -N\widehat{R}_{\mathrm{CE}}(\theta)$. A monotone transform and a negative constant do not move the argmax, so the two optimisation problems have the same solutions. Without independence the likelihood would not factorise and the sum of per-example losses would no longer be the log-likelihood.

Exercises 2, Q4(c)–(d) are the example and the key insight above; the three-step argument (roles of the two losses; cross-entropy is proper, $H + \mathrm{KL}$; good probabilities serve every decision rule) is what the rubric wants, plus the remark that four examples cannot rank population zero–one error.

Exercises 2, Q7(a)–(c). *Data* $(-1,1),(0,0),(1,1)$*;* $f_1 = x^2$*,* $f_{2,\alpha} = x^2 + \alpha(x+1)x(x-1)$*. Show zero training error for all; compare at* $x = \frac{1}{2}$*; why can ERM not choose?* The extra term contains a factor $(x+1)$, x or $(x-1)$, so it is 0 at each training input and $f_{2,\alpha}(x^{(i)}) = y^{(i)}$ for every α. At $x = \frac{1}{2}$: $f_1 = \frac{1}{4}$, $(x+1)x(x-1) = -\frac{3}{8}$, so $f_{2,\alpha}(\frac{1}{2}) = \frac{1}{4} - \frac{3}{8}\alpha$, any real number. ERM ranks by training risk only; all tie at the minimum 0, so the choice between them is made by something other than the data: a restricted $\mathcal{H}$ (degree ≤ 2 leaves only x^2), a penalty on $\|\theta\|^2 = 1 + 2\alpha^2$ (picks $\alpha = 0$), or validation points at new x.

New (derive). *Show that the gradient of the empirical cross-entropy of logistic regression vanishes only when the residuals* $\hat{p}_\theta(x^{(i)}) - y^{(i)}$ *are orthogonal to every feature and sum to zero; interpret.* From the formulas above, $\nabla_w \widehat{R} = 0$ iff $\sum_i (\hat{p}^{(i)} - y^{(i)}) x_j^{(i)} = 0$ for each j, and $\partial \widehat{R}/\partial b = 0$ iff $\sum_i \hat{p}^{(i)} = \sum_i y^{(i)}$. The second says the fitted model is calibrated on average: predicted positives equal observed positives on the training set. The first says no feature carries leftover linear information about the residual. These are the logistic analogue of the normal equations; they have no closed form because $\hat{p}$ is nonlinear in θ, hence iterative optimisation.

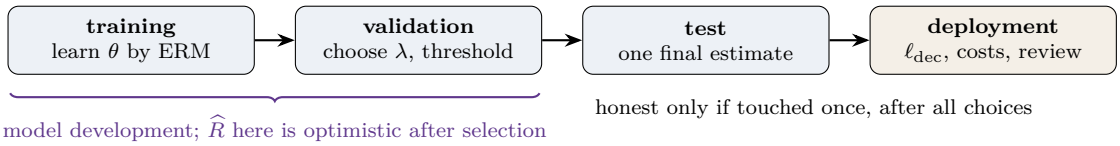
6 Evaluation Basics

Training, validation and test sets; cross-validation

The **training set** learns θ through ℓ_{train}. The **validation set** compares the fitted configurations and selects hyperparameters: for each candidate λ, $\hat{\theta}_\lambda = \arg\min_\theta \widehat{R}_{\mathrm{train}}(\theta;\lambda)$, then $\hat{\lambda} = \arg\min_\lambda \widehat{R}_{\mathrm{val}}(\hat{\theta}_\lambda;\lambda)$. The **test set** is used once, after every choice is fixed, to estimate final performance with the measure the application cares about. **K-fold cross-validation** splits the development data into K folds, trains K times leaving one fold out, averages the K validation risks, selects $\hat{\lambda}$, then refits on all development data; the test set stays untouched.

Why a subset stops being informative the moment it is used to choose

A held-out risk is honest for a *fixed* model, by the same argument as in Section 4. Once we select the candidate with the lowest validation risk, that minimum is optimistic, by the same mechanism that makes training risk optimistic after fitting (Section 5, with the validation set playing the role of the training set and the candidates the role of Θ). Consulting the test set during development turns it into a second validation set. Anything *computed* from data (scaling constants, vocabulary, imputed values, selected features) is a fitted quantity: compute it on the training portion and apply it to the rest, or it leaks. Grouped data (near-duplicate flows of one attack, several images of one patient) must stay inside one subset, and time-ordered data split by time, or the test set contains copies of the training set.



Tutorial 1, Q4(c)–(d): the metrics and the all-benign baseline

10,000 flows, 100 attacks; TP = 70, FP = 180, FN = 30, TN = 9720. Accuracy = $(70 + 9720)/10{,}000 = 0.979$. Precision = $70/(70 + 180) = 0.28$. Recall = $70/100 = 0.70$. $F_1 = 2 \cdot 0.28 \cdot 0.70/(0.28 + 0.70) = 0.40$. The rule "everything is benign" has accuracy 0.99, higher, with recall 0: accuracy is dominated by the 99% majority class and cannot see the attacks. Report recall, precision, false alarms per day, and the expected cost at the operating threshold.

Tutorial drill

Exercises 2, Q10(a). *Near-duplicate flows per attack, standardisation before the split, random row split,* 500 *configurations chosen by test accuracy, threshold* 0.5 *with* $C_{\mathrm{FN}} = 100\, C_{\mathrm{FP}}$. *Give four reasons the reported accuracy misleads.* (1) Duplicates split at random: the test set contains copies of training flows, so the score measures recognition of seen attacks. (2) Standardising before the split leaks test statistics into the representation. (3) Picking the best of 500 by test accuracy makes the test set a validation set; the best of 500 noisy numbers is strongly optimistic. (4) Accuracy on a rare class can be high while most attacks are missed. (5) The Bayes threshold is $C_{\mathrm{FP}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}}) = 1/101 \approx 0.01$; at 0.5 every attack with posterior in $(0.01, 0.5)$ is left unflagged. Fix: deduplicate, split by episode and time, fit scaling inside the training folds, choose the configuration *and* the threshold on validation, evaluate once.

Exercises 2, Q10(c). *Separate training loss, validation criterion, test metric, deployment loss.* Training: cross-entropy (estimates $\hat{p}$). Validation: expected cost per flow with $C_{\mathrm{FP}} = 1$, $C_{\mathrm{FN}} = 100$ at the candidate threshold, or recall at a false-alarm budget (chooses configuration and threshold). Test: the same criterion plus recall, precision, false-positive rate, once. Deployment: alert when the calibrated posterior exceeds the chosen threshold; the threshold stays adjustable because the decision policy is separate from the learned probabilities.

New (counterexample). *Show with three folds that a single validation split can select the wrong* k *for kNN.* Fold errors for $k = 1$: $0.40, 0.20, 0.60$ (mean 0.40); for $k = 3$: $0.30, 0.30, 0.30$ (mean 0.30). Validating on fold 2 alone picks $k = 1$; cross-validation picks $k = 3$. Averaging over folds removes the dependence on one accidental split; refit with $\hat{k} = 3$ on all development data afterwards, and remember that the selected 0.30 is itself optimistic: the test set gives the number to report.

The mistakes that cost points

- Inverting the threshold. C_{FP} is on top: $C_{\mathrm{FP}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$. Expensive misses must give a *small* threshold ($2/(2+8) = 0.2$, not 0.8); $C_{\mathrm{FP}}/C_{\mathrm{FN}}$ is the odds, not the threshold.
- "Predict the most likely value." That is the Bayes rule for zero–one loss only. Squared loss wants the mean (which may never occur), absolute the median, c_u versus c_o the $c_u/(c_u+c_o)$-quantile; write the derivative of the conditional risk, not the name.
- Reading "B has lower cross-entropy but more errors" as an argument against cross-entropy. Training loss estimates probabilities, decision loss acts on them; cross-entropy is the loss minimised by the true posterior, and calibrated probabilities serve every threshold and the reject band.
- Scaling, selecting features or picking k, w, the threshold or the configuration with test data. Any quantity computed from data is fitted; the minimum over candidates is optimistic; the test set is touched once.

Chapter 2 in five lines

1. Objects become $x \in \mathbb{R}^d$ by human choices (one-hot for categories); two objects with the same x and different labels create a Bayes error $1 - \max_c \mathbb{P}(Y = c \mid x)$ that no model can beat.
2. Known distribution: $f^*(x) = \arg\min_a \mathbb{E}[\ell_{\mathrm{dec}}(Y, a) \mid X = x]$. Zero–one $\to$ argmax; costs $\to$ positive iff $p(x) > C_{\mathrm{FP}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$; review at cost $C_{\mathrm{R}} \to$ negative below $C_{\mathrm{R}}/C_{\mathrm{FN}}$, positive above $1 - C_{\mathrm{R}}/C_{\mathrm{FP}}$, review between (band exists iff $C_{\mathrm{R}} < C_{\mathrm{FP}}C_{\mathrm{FN}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$).
3. Regression: squared $\to$ mean, absolute $\to$ median, under-cost c_u / over-cost $c_o \to$ quantile $c_u/(c_u + c_o)$; exponential scale 4: 4, $4 \ln 2 = 2.77$, $4 \ln 5 = 6.44$.
4. Finite data: $\widehat{R}$ is honest for a fixed rule, optimistic after selection. kNN plugs a local average into the Bayes rule and inherits the metric (Tutorial 1, Q3: A raw, B scaled, $w^* = 1.654$); $\mathbb{P}(\text{neighbour within } 0.1) = 1 - (1 - 10^{-d})^n$ collapses at $d = 4$ for $n = 1000$.
5. ERM over $\mathcal{H} = \{f_\theta\}$; cross-entropy $-\log \hat{p}_{\theta,y}(x)$ equals maximum likelihood, its expectation is $H + \mathrm{KL}$ (minimised by the truth), and for logistic regression $\nabla_w \widehat{R} = \frac{1}{N} \sum (\hat{p}^{(i)} - y^{(i)}) x^{(i)}$. Train / validate / test; touch the test set once.